쿼리 파싱 및 AST 변환 상세 분석 (Query Parsing and AST Transformation)

파일 개요

- **파일 경로**: `crates/proof-of-sql/src/sql/parse/query_expr.rs`
- **주요 역할**: SQL 문법 파싱과 내부 표현으로의 변환
- **핵심 기능**: 중간 AST를 증명 가능한 실행 계획으로 변환

모듈 구조 및 의존성

주요 Import 분석

```
use super::{EnrichedExpr, FilterExecBuilder, QueryContextBuilder};
use crate::{
base::database::SchemaAccessor,
sql::{
parse::ConversionResult,
postprocessing::{
GroupByPostprocessing, OrderByPostprocessing, OwnedTablePostprocessing,
SelectPostprocessing, SlicePostprocessing,
},
proof_plans::{DynProofPlan, GroupByExec},
```

```
},

};

use proof_of_sql_parser::{intermediate_ast::SetExpression, SelectStatement};
```

설명:

- **proof_of_sql_parser**: 외부 파서 라이브러리에서 중간 AST를 가져옵니다
- **postprocessing**: 증명 후 처리 단계들 (정렬, 슬라이싱, 그룹화 등)
- **proof_plans**: 증명 가능한 실행 계획들

핵심 데이터 구조

1. QueryExpr 구조체

```
#[derive(PartialEq, Serialize, Deserialize)]

/// A `QueryExpr` represents a Proof of SQL query that can be executed against a

/// It consists of a `DynProofPlan` for provable components and a vector of `Own

pub  struct QueryExpr {

proof_expr: DynProofPlan,

postprocessing: Vec<OwnedTablePostprocessing>,

}
```

설명:

- **proof_expr**: 영지식 증명이 가능한 부분 (WHERE, SELECT, GROUP BY 등)
- **postprocessing**: 증명 후 처리가 필요한 부분 (ORDER BY, LIMIT 등)

- **분리 설계**: 증명 가능한 부분과 그렇지 않은 부분을 명확히 구분

2. Debug 구현

```
impl fmt::Debug for QueryExpr {
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
write!(
f,
"QueryExpr \n[{:#?},\n{:#?}\n]",
self.proof_expr, self.postprocessing
)
}
}
```

설명: 디버깅을 위한 가독성 있는 출력 형식을 제공합니다.

파싱 과정 분석 (try_new 함수)

1. 함수 시그니처 및 초기 설정

```
pub  fn try_new(
ast: SelectStatement,
default_schema: Ident,
schema_accessor: &dyn SchemaAccessor,
```

```
) -> ConversionResult<Self> {
```

설명:

- **ast**: 파서에서 생성된 중간 AST
- **default_schema**: 기본 스키마 이름
- **schema_accessor**: 데이터베이스 스키마 정보 접근자

2. QueryContext 구축

```
let context = match *ast.expr {
SetExpression::Query {
result_exprs,
from,
where_expr,
group_by,
} => QueryContextBuilder::new(schema_accessor)
.visit_table_expr(&from, default_schema)
.visit_group_by_exprs(group_by.into_iter().map(Ident::from).collect())?
.visit_result_exprs(result_exprs)?
.visit_where_expr(where_expr)?
.visit_order_by_exprs(ast.order_by.into_iter().map(Into::into).collect())?
.visit_slice_expr(ast.slice)
.build()?,
};
```

설명:

- **빌더 패턴**: QueryContextBuilder를 사용하여 단계별로 컨텍스트를 구축합니다
- **방문자 패턴**: 각 SQL 구성 요소를 순차적으로 방문하여 처리합니다
- **타입 검사**: 각 단계에서 데이터 타입 호환성을 검사합니다

3. 후처리 단계 결정

```
let result_aliased_exprs = context.get_aliased_result_exprs()?.to_vec();

let group_by = context.get_group_by_exprs();

// Figure out the basic postprocessing steps.

let  mut postprocessing = vec![];

let order_bys = context.get_order_by_exprs();

if !order_bys.is_empty() {

postprocessing.push(OwnedTablePostprocessing::new_order_by(

OrderByPostprocessing::new(order_bys.to_vec()),

));

}

if  let Some(slice) = context.get_slice_expr() {

postprocessing.push(OwnedTablePostprocessing::new_slice(

SlicePostprocessing::new(Some(slice.number_rows), Some(slice.offset_value)),

));

}
```

설명:

- **ORDER BY**: 정렬은 증명 후 처리로 분류됩니다
- **LIMIT/OFFSET**: 슬라이싱도 후처리 단계입니다
- **순서 중요**: 후처리 단계들의 순서가 결과에 영향을 미칩니다

4. 집계 함수 처리 분기

```
if context.has_agg() {
if  let Some(group_by_expr) = Option::<GroupByExec>::try_from(&context)? {
Ok(Self {
proof_expr: DynProofPlan::GroupBy(group_by_expr),
postprocessing,
})
} else {
// 집계 함수가 있지만 GROUP BY가 없는 경우
let raw_enriched_exprs = result_aliased_exprs
.iter()
.map(|aliased_expr| EnrichedExpr {
residue_expression: aliased_expr.clone(),
dyn_proof_expr: None,
})
.collect::<Vec<_>>();
```

설명:

- **집계 함수 감지**: context.has_agg()로 집계 함수 사용 여부를 확인합니다

- **GROUP BY 분기**: GROUP BY가 있으면 GroupByExec, 없으면 특별 처리합니다
- **EnrichedExpr**: 표현식을 증명 가능한 부분과 후처리 부분으로 분리합니다

5. 필터 실행 계획 생성 (집계 없는 경우)

```
} else {
// No group by, so we need to do a filter.
let column_mapping = context.get_column_mapping();
let enriched_exprs = result_aliased_exprs
.iter()
.map(|aliased_expr| EnrichedExpr::new(aliased_expr.clone(), &column_mapping))
.collect::<Vec<_>>();
let select_exprs = enriched_exprs
.iter()
.map(|enriched_expr| enriched_expr.residue_expression.clone())
.collect::<Vec<_>>();
let filter = FilterExecBuilder::new(context.get_column_mapping())
.add_table_expr(context.get_table_ref().clone())
.add_where_expr(context.get_where_expr().clone())?
.add_result_columns(&enriched_exprs)
.build();
```

설명:

- **단순 쿼리**: 집계 함수가 없는 경우 FilterExec을 사용합니다

- **컬럼 매핑**: 컬럼 이름을 ColumnRef로 매핑합니다
- **표현식 분석**: 각 SELECT 표현식이 증명 가능한지 판단합니다

QueryContextBuilder 상세 분석

1. 구조체 정의

```
pub struct QueryContextBuilder<'a> {
context: QueryContext,
schema_accessor: &'a dyn SchemaAccessor,
}
```

설명:

- **context**: 구축 중인 쿼리 컨텍스트
- **schema_accessor**: 스키마 정보에 접근하기 위한 인터페이스

2. 테이블 표현식 방문

```
pub fn visit_table_expr(
mut self,
table_expr: &[Box<TableExpression>],
default_schema: Ident,
) -> Self {
assert_eq!(table_expr.len(), 1);
```

```
match *table_expr[0] {
TableExpression::Named { table, schema } => {
let actual_schema = schema.map_or(default_schema, Ident::from);
let table_ref = TableRef::from_idents(Some(actual_schema), Ident::from(table));
self.context.set_table_ref(table_ref);
}
}
self
}
```

설명:

- **단일 테이블**: 현재는 하나의 테이블만 지원합니다 (JOIN 미지원)
- **스키마 처리**: 명시적 스키마가 없으면 기본 스키마를 사용합니다
- **TableRef 생성**: 스키마와 테이블 이름으로 참조를 생성합니다

3. WHERE 절 표현식 방문

```
pub fn visit_where_expr(
mut self,
mut where_expr: Option<Box<Expression>>,
) -> ConversionResult<Self> {
if let Some(expr) = where_expr.as_deref_mut() {
self.visit_expr(expr)?;
}
```

```
self.context.set_where_expr(where_expr);

Ok(self)

}
```

설명:

- **선택적 처리**: WHERE 절이 없을 수도 있습니다
- **표현식 방문**: 재귀적으로 표현식을 방문하여 타입 검사를 수행합니다

4. 이진 연산자 처리

```
fn visit_binary_expr(

&mut  self,

op: &BinaryOperator,

left: &Expression,

right: &Expression,

) -> ConversionResult<ColumnType> {

let left_dtype = self.visit_expr(left)?;

let right_dtype = self.visit_expr(right)?;

match *op {

BinaryOperator::And | BinaryOperator::Or => can_and_or_types(left_dtype, right_d

.then_some(ColumnType::Boolean)

.ok_or(ConversionError::DataTypeMismatch {

left_type: left_dtype.to_string(),

right_type: right_dtype.to_string(),
```

```
}),
BinaryOperator::Eq => Ok(try_equals_types_with_scaling(left_dtype, right_dtype)
.map(|()| ColumnType::Boolean)?),
BinaryOperator::Plus | BinaryOperator::Minus => Ok(
try_add_subtract_column_types_with_scaling(left_dtype, right_dtype)?,
),
// ... 기타 연산자들
}
}
```

설명:

- **타입 호환성**: 각 연산자에 대해 피연산자 타입의 호환성을 검사합니다
- **타입 승격**: 필요시 타입을 더 큰 타입으로 승격시킵니다 (예: Int → BigInt)
- **스케일링**: Decimal 타입의 경우 스케일을 조정합니다

5. 집계 함수 처리

```
fn visit_agg_expr(
&mut  self,
op: AggregationOperator,
expr: &Expression,
) -> ConversionResult<ColumnType> {
self.context.set_in_agg_scope(true)?;
```

```
let expr_dtype = self.visit_expr(expr)?;


// We only support sum/max/min aggregations on numeric columns.
if op != AggregationOperator::Count && expr_dtype == ColumnType::VarChar {
return Err(ConversionError::non_numeric_expr_in_agg(
expr_dtype.to_string(),
op.to_string(),
));
}


self.context.set_in_agg_scope(false)?;


// Count aggregation always results in an integer type
if op == AggregationOperator::Count {
Ok(ColumnType::BigInt)
} else {
Ok(expr_dtype)
}
}
```

설명:

- **집계 스코프**: 집계 함수 내부임을 표시하여 중첩 집계를 방지합니다
- **타입 제한**: SUM, MAX, MIN은 숫자 타입에만 적용 가능합니다

- **COUNT 특별 처리**: COUNT는 항상 BigInt를 반환합니다

FilterExecBuilder 분석

1. 빌더 구조체

```
pub struct FilterExecBuilder {
table_expr: Option<TableExpr>,
where_expr: Option<DynProofExpr>,
filter_result_expr_list: Vec<AliasedDynProofExpr>,
column_mapping: IndexMap<Ident, ColumnRef>,
}
```

설명:

- **단계별 구축**: 각 구성 요소를 단계별로 추가합니다
- **타입 안전성**: Option 타입으로 필수 구성 요소의 누락을 방지합니다

2. WHERE 절 추가

```
pub fn add_where_expr(
mut self,
where_expr: Option<Box<Expression>>,
) -> Result<Self, ConversionError> {
self.where_expr = WhereExprBuilder::new(&self.column_mapping).build(where_expr)?
Ok(self)
```

```
}
```

설명:

- **WhereExprBuilder**: 전용 빌더로 WHERE 절을 DynProofExpr로 변환합니다
- **컬럼 매핑 사용**: 컬럼 이름을 실제 ColumnRef로 해석합니다

3. 결과 컬럼 처리

```
pub fn add_result_columns(mut self, columns: &[EnrichedExpr]) -> Self {
// If a column is provable, add it to the filter result expression list
// If at least one column is non-provable, add all columns from the column mappi
let mut has_nonprovable_column = false;
for enriched_expr in columns {
if let Some(plan) = &enriched_expr.dyn_proof_expr {
self.filter_result_expr_list.push(AliasedDynProofExpr {
expr: plan.clone(),
alias: enriched_expr.residue_expression.alias.into(),
});
} else {
has_nonprovable_column = true;
}
}

if has_nonprovable_column {
```

```
// Has to keep them sorted to have deterministic order for tests

for alias in  self.column_mapping.keys().sorted() {

let column_ref = self.column_mapping.get(alias).unwrap();

self.filter_result_expr_list.push(AliasedDynProofExpr {

expr: DynProofExpr::new_column(column_ref.clone()),

alias: alias.clone(),

});

}

}

self

}
```

설명:

- **증명 가능성 판단**: 각 표현식이 증명 가능한지 확인합니다
- **폴백 전략**: 하나라도 증명 불가능하면 모든 컬럼을 포함합니다
- **결정적 순서**: 테스트의 일관성을 위해 정렬된 순서를 유지합니다

타입 검사 및 최적화 과정

1. 타입 호환성 검사

- **이진 연산**: 좌우 피연산자의 타입 호환성 검사
- **집계 함수**: 적절한 데이터 타입에만 적용 가능
- **비교 연산**: 동일하거나 호환 가능한 타입 간에만 비교 가능

2. 타입 승격 (Type Promotion)

- **숫자 타입**: 작은 타입을 큰 타입으로 자동 승격
- **Decimal 스케일링**: 정밀도와 스케일을 적절히 조정
- **Boolean 연산**: AND, OR 연산은 Boolean 타입만 허용

3. 최적화 기법

- **상수 폴딩**: 컴파일 타임에 상수 표현식 계산
- **표현식 분리**: 증명 가능한 부분과 후처리 부분 분리
- **메모리 효율성**: 필요한 컬럼만 로드하여 메모리 사용량 최적화

SQL에서 증명 가능한 형태로의 변환 과정

1. 파싱 단계

1. **SQL 문자열** → **sqlparser AST** (외부 라이브러리)
2. **sqlparser AST** → **중간 AST** (proof_of_sql_parser)
3. **중간 AST** → **QueryExpr** (이 모듈)

2. 변환 단계

1. **테이블 참조 해석**: 스키마와 테이블 이름을 TableRef로 변환
2. **컬럼 참조 해석**: 컬럼 이름을 ColumnRef로 변환 (타입 정보 포함)
3. **표현식 변환**: SQL 표현식을 DynProofExpr로 변환
4. **실행 계획 생성**: FilterExec 또는 GroupByExec 생성

3. 증명 준비

1. **MLE 변환**: 테이블 데이터를 다선형 확장으로 표현

2. **다항식 구성**: SQL 연산을 다항식 제약 조건으로 변환

3. **Sumcheck 준비**: 증명에 필요한 다항식들을 구성

이 과정을 통해 복잡한 SQL 쿼리가 영지식 증명 가능한 형태로 체계적으로 변환되며, 타입 안전성과 정확성이 보장됩니다.